

1 / User-OAuth Service endpoints

Owner: Shakti. Local issuer <http://localhost:9000>; Gateway <http://localhost:8080>; React <http://localhost:5173>. AWS deployment replaces these with exact HTTPS origins. OAuth protocol calls go to issuer; /api business calls go through Gateway.

Method / path	Input and expected result
GET /oauth2/authorize	Browser code + PKCE S256; client_id technotes-web; redirect_uri /auth/callback on UI origin; scope, state, nonce, challenge. Redirects with code/state.
POST /oauth2/token	Form code exchange with code_verifier and same redirect_uri. JSON access_token, token_type=Bearer, expires_in. Browser CORS enabled; no client secret.
GET /oauth2/jwks	Public JWK Set. Notes validates token signature and issuer.
GET /api/v1/users/me	Bearer + profile.read. Direct JSON id, displayName, email, roles, status. UI uses this for admin workspace.

Issuer configuration and token contract

Provision one owner account privately; public signup is not required. Issue roles ADMIN (and AUTHOR if your Notes policy requires), scopes notes.read, notes.write, notes.review, taxonomy.write, profile.read. JWT: iss exact HTTPS issuer, immutable UUID sub, aud=[technotes-api], scope as space-separated string, roles array, iat/nbf/exp. Stable signing key and persisted OAuth state; no long-lived secret in UI.

Required checks

Correct code+PKCE works; replay/wrong verifier fails; token endpoint allows only configured UI origin; Notes rejects wrong signature/issuer/audience/expired token and ID token. /users/me returns 401 without valid access token. In AWS, register exact <https://technotes.co.in/auth/callback> and use a stable issuer such as <https://auth.technotes.co.in>.

Shared response rules

JSON resource responses are direct objects. List responses use {items,page,size,totalElements,totalPages}. Business errors use {timestamp,status,code,message,path,traceId,fieldErrors}. Gateway forwards Authorization and If-Match; CORS exposes ETag and Location. IDs UUID strings; dates ISO-8601 UTC.

2 / Notes Service endpoints

Owner: Developer 2. Notes uses MongoDB, validates issuer JWT, derives authorId from sub. Public readers never need login. Current v1 Notes spec governs DTO validation and review workflow; exceptions below require explicit review.

Method / Gateway path	Input / output
GET /api/v1/public/categories	rootOnly=true or parentId, page=0,size=100. Page of active category nodes: id,name,slug,parentId,level,active,sortOrder.
GET /api/v1/public/notes	page=0,size=100,sort=publishedAt,desc; optional primaryCategoryId. Page of published PUBLIC cards: id,slug,title,summary,primaryCategoryId,categoryName,tags,publishedAt; no Markdown.
GET /api/v1/public/notes/{slug}	Published PUBLIC snapshot: id,slug,title,summary,contentMarkdown,primaryCategoryId,categoryName,tags,revisionNumber,publishedAt. Hidden/unknown -> 404.
POST /api/v1/categories	Bearer ADMIN + taxonomy.write; name,slug,parentId=null,sortOrder=0. -> 201 CategoryResponse.
GET /api/v1/notes	view=mine,page=0,size=100,sort=updatedAt,desc; Bearer notes.read. Page of own id,title,slug,summary,primaryCategoryId,tags,status,visibility,version,updatedAt.
GET /api/v1/notes/{id}	Owner/ADMIN + notes.read. NoteResponse with contentMarkdown and ETag "note-{id}-vN".
POST /api/v1/notes	Bearer notes.write. title,summary,contentMarkdown,primaryCategoryId,tags,visibility=PUBLIC. -> 201 NoteResponse DRAFT, ETag.
PATCH /api/v1/notes/{id}	Editable fields + If-Match; own DRAFT -> 200 updated NoteResponse + new ETag.
POST /api/v1/notes/{id}/submit	No body + If-Match; DRAFT -> IN_REVIEW. Updated NoteResponse + ETag.
POST /api/v1/notes/{id}/publish	No body + If-Match; ADMIN + notes.review; IN_REVIEW -> PUBLISHED. Immutable snapshot, updated NoteResponse + ETag.

Minimum sequence and one-owner policy

Create root category -> create DRAFT -> edit -> submit -> publish -> anonymous public list/detail. PUBLIC visibility alone never exposes a draft. For today, approve an explicit audited ADMIN self-publish exception to the existing reviewer self-approval rule; do not silently bypass it. Mongo replica-set transaction keeps published snapshot/pointer atomic.

Error and deployment gate

401 missing/invalid token; 403 role/scope; 404 hidden/unknown; 409 invalid transition; 412 stale ETag; 428 missing If-Match. Tests prove draft/PRIVATE/archived never leak, author isolation, admin publish audit and persistence after restart. Complete JSON examples and UI configuration are in TechNotes_First_Live_Release_API_Contract.md. Build and integration remain to be verified against live services.